Clustbench: A Reproducible, Trajectory-Instrumented, Self-Extending Benchmark for Clustering Algorithms

Ashail Maharaj
https://github.com/Ashail33/clustbench

October 2026

Abstract

Clustering algorithms are hard to compare. Published evaluations differ in datasets, sample sizes, metrics, hyperparameters and hardware, and few report runtime or memory in a reproducible way. We present *Clustbench*, an open-source benchmark harness that makes the comparison mechanical. A YAML configuration expands into a Cartesian grid of tasks over synthetic, real-world, time-series, graph and adversarial datasets with known ground truth. Every registered algorithm, whether native Python or any executable speaking a four-field JSON protocol, runs on the same data, and each run is recorded as one schema-validated row holding external validity (ARI, NMI, and inlier-only variants), internal validity and resource use. Clustbench adds three things that existing clustering benchmarks do not provide together: a *trajectory layer* that logs every step of an iterative algorithm as a $(\text{state}, \text{action}, \text{cost}, \Delta\text{cost})$ record; machine-readable *algorithm cards* with theoretical predictions; and a *self-extending* layer of learned algorithm selectors that retrain on the benchmark's own output. We pair each threat to the validity of such a benchmark with a control in the harness: post-hoc tests and configuration-level confidence intervals, five seeds, repeated timing of the algorithm call alone, checked data provenance, inlier-only scoring, run isolation with time and memory limits, large-n sweeps, and leave-one-dataset-family-out evaluation of the selectors. On the controlled release (61 algorithms, 20 dataset families, 140 tasks, 8,539 runs) the Friedman test rejects equal performance ($\chi^2 = 2700$), yet a Nemenyi critical difference cannot separate the top 15 algorithms. The best selector beats the best single algorithm, an EM-trained graph-Laplacian mixture, by 0.05 ARI in-distribution. It loses 0.03–0.07 ARI on held-out dataset families, where no selector differs significantly from that algorithm. Two wider runs show where the selectors' advantage ends. A run-isolated big-data sweep to $n = 200{,}000$, $d = 500$ and $k = 200$ measures each algorithm's growth exponents and records 141 runs that hit a limit. An every-algorithm-on-every-dataset run (63 algorithms, 36 dataset families, 11,328 runs) finds the selectors at 0.31–0.35 ARI on curved shapes, against 0.74–0.86 for graph-based methods. On real data they lead only on the datasets in their training table; on seven real datasets they have never seen, they tie the best single methods. The controls also corrected four earlier results: the "real" time series had silently been synthetic, BIRCH scales quadratically beyond $n \approx 3{,}000$ in 10 dimensions, the treatment of injected outliers reorders the leaderboard on contaminated data, and an internal validity index needed 16 GB at $d = 500$.

1 Introduction

Clustering has no single correct objective [1], so the practical question *which algorithm should I use on this data?* is answered empirically. The empirical literature is fragmented. Each paper picks its own datasets, scale, hyperparameter budget, metrics and hardware, and wall time and memory are usually reported anecdotally if at all. A white paper on benchmarking in cluster analysis [2] names these problems and calls for neutral, reproducible comparisons.

The author's MEng research assignment [3] reviewed big-data clustering methods, proposed a four-way taxonomy (sampling/partitioning, ensemble, parallel/distributed and incremental methods), and benchmarked one representative of each category: S^5C [4], PWCC [5], MapReduce-style parallel k-means [6] and mini-batch k-means [7]. Its closing recommendation was a *centralised database for clustering research* that would let practitioners find the right model for their data and infrastructure. That study's experiments were limited by compute: the full factorial grid exceeded the available notebook quota. Clustbench turns the recommendation into software and keeps the grid cheap to re-run.

Contributions.

1. **A language-agnostic harness** (§3) that expands a declarative configuration into a task grid, runs any number of algorithms on identical data, and writes one tidy, schema-validated record per run together with resource measurements, average ranks and a Friedman test.
2. **A trajectory layer** (§3.5) that records the step-by-step optimisation of iterative algorithms as state–action–reward tuples. To our knowledge this is the first clustering benchmark to treat the *search process* as a first-class output.
3. **Algorithm cards, a gap finder and a mutator** (§3.6) that encode each algorithm's mechanism, inductive biases and complexity in machine-readable form, predict cost and quality from them, and search the fingerprint space for regimes no registered algorithm covers.
4. **A benchmark suite** (§4) of more than thirty dataset generators across convex, non-convex, heterogeneous-density, outlier-heavy, low-rank high-dimensional, real tabular, time-series, graph and deliberately adversarial regimes, and a registry of 59 algorithm implementations.
5. **Validity controls** (§3.7) for each threat we identified in our own earlier results: post-hoc statistics, seed count, timing method, data provenance, outlier scoring, run isolation, scale, and selector leakage.
6. **An empirical study** (§5) with in-distribution, leave-one-dataset-family-out, out-of-distribution, big-data and every-algorithm-on-every-dataset evaluations, including the earlier results that the controls overturned (§6).

A companion paper [8] describes the new algorithms developed with the harness. This paper covers the harness, the suite and what the benchmark measures.

2 Related work

Clustering benchmarks. Van Mechelen et al. [2] set out principles for neutral benchmarking in cluster analysis. Gagolewski [9] proposed a framework and curated battery of low-dimensional benchmark datasets with reference partitions, distributed as a Python package that is *also* named `clustbench`. The two projects are independent and complementary: Gagolewski's work is a dataset battery with reference labels, while ours is a run harness with resource measurement, trajectory capture and learned selection. Gagolewski's batteries could be loaded through our dataset interface. Fahad et al. [10] gave an empirical taxonomy of big-data clustering algorithms that the author's review built on [3].

Synthetic data with ground truth. MDCGen [11] generates multidimensional clustering datasets with controllable outliers, noise and density. Clustbench reimplements an MDCGen-style generator and adds shape generators (moons, circles, spirals, Swiss roll, rings), heavy-tailed and imbalanced mixtures, and adversarial families.

Statistical comparison. We follow Demšar's recommendation [12] of average ranks with the Friedman test [13] for comparing many algorithms over many datasets.

Algorithm selection and meta-learning. Choosing an algorithm from dataset features is Rice's algorithm selection problem [14]. It has been studied as meta-learning in general [15, 16] and for clustering specifically [17, 18], including recent AutoML systems for clustering that select algorithms from meta-features, dataset similarity or learned embeddings [19–21]. It is also used for warm-starting in AutoML [22]. Clustbench's learned routers are instances of this idea. The contribution here is the infrastructure that makes them self-extending: the router training set *is* the benchmark output, so it grows whenever the benchmark runs.

3 Design

3.1 Task grid

A benchmark is a YAML file. Each dataset entry lists value sets for sample size n, dimension d, target cluster count k, compactness and generator-specific knobs (outlier count, noise points, density, outlier extremity). The harness takes the Cartesian product of every list and crosses it with a list of seeds:

```
datasets:
  - id: mdcgen
    n_samples: [500]
    n_features: [10]
    k_targets: [3, 5]
    outliers: [0, 100]
algorithms:
  - {name: kmeans, kind: python, entry: kmeans, params: {n_init: 3}}
  - {name: mine, kind: external, entry: ./bin/run_my_algo}
seeds: [1, 2, 3]
```

A task is uniquely identified by (dataset, n, d, k, compactness, outliers, noise, density, seed). Every algorithm sees the identical array for a given task.

3.2 Algorithm interface

Native algorithms subclass a small abstract base class, register themselves under a string key, and implement `fit_predict(X, k)`, which returns an `AlgoResult` with integer labels, an optional dictionary of algorithm-specific extras, and an optional trajectory. External algorithms in any language are supported through a JSON-over-stdin protocol. The harness writes the task to a `.npy` file and sends

```
{"data_path": "...", "k": 5, "params": {...},
 "artifacts_dir": "...", "labels_path": "..."}
```

on standard input. The executable writes integer labels to `labels_path` and may print a JSON object of extras to standard output. That is the entire protocol.

3.3 Records and resource measurement

Each (algorithm, task) pair produces one record validated by a Pydantic schema (Appendix A). Resource use is measured with `psutil` around the `fit_predict` call *only*: wall-clock time, resident-set-size delta, user and system CPU time, and read/write bytes where the platform exposes them. Label saving and scoring happen outside the timed window (§3.7). Metrics are computed by one function shared by all algorithms: ARI [23] and NMI [24] against ground truth; silhouette [25] (sub-sampled above 5,000 points), Davies–Bouldin [26] and Dunn [27] as internal indices; plus compactness and separation. Because generators label injected outliers -1, we also report `ari_inliers` and `nmi_inliers`, computed on ground-truth inliers only. The results table is flat (one column per metric), so a leaderboard is a one-line query in pandas or DuckDB. A

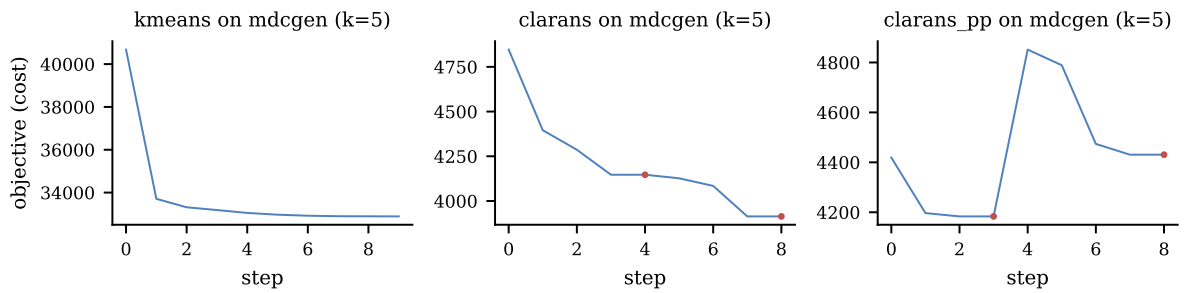


Figure 1: Trajectory records for one MDCGen task (n=500, d=10, k=5, 100 injected outliers, seed 1). k-means descends monotonically in a few EM steps; CLARANS and its k-means++-seeded variant take many local swap steps, and red dots mark rejected proposals.

crashing algorithm is logged and skipped and does not abort the sweep. Logged failures are not harmless, though: a missing row silently drops out of an algorithm's mean. Section 6 lists the root causes we found this way.

3.4 Aggregation

For each metric, algorithms are ranked within every task (rank 1 = best, ties averaged). Ranks are averaged across tasks, and the Friedman statistic is computed over the tasks on which every algorithm produced a result [12]. Because the Friedman test is nearly always significant with dozens of algorithms, the summary also carries post-hoc statistics (§3.7). The summary is written to `summary.json` and also recomputed in the browser by the dashboard, so adding an algorithm never requires changing analysis code.

3.5 The trajectory layer

Iterative clustering algorithms are sequential decision processes. At each step they observe a state (centroids, medoids, an assignment, a mixture's parameters), take an action (reassign, move a centroid, swap a medoid, merge two clusters), and observe a change in objective. Clustbench lets an algorithm return a list of `Step` objects that are persisted as a Parquet trajectory table with one row per step:

$$(\mathit{run_id}, \mathit{task}, t,\ s_t,\ a_t,\ c_t,\ \Delta c_t = c_t - c_{t-1},\ \mathit{accepted}_t).$$

Here s_t is a numeric state snapshot, a_t is a typed action (e.g. `{"type": "swap", "out": 17, "in": 42}`), c_t is the scalar objective, and the boolean records whether a proposed action was accepted (local-search methods such as CLARANS [28] reject most proposals). The negative of Δc_t is a natural reward. Concatenated across runs, trajectories are a dataset for value models over $(s, a) \mapsto \Delta c$, for learned proposal distributions, and for imitation learning of clustering policies. The companion paper [8] trains such a policy. In the current release, 45 of the 61 benchmarked algorithms emit trajectories, and the in-distribution run stores 6,300 of them. Figure 1 shows three.

3.6 Algorithm cards, gap finder and mutator

Each registered algorithm has an *algorithm card*: its taxonomy family, a set of mechanism tags (e.g. `kpp_init`, `graph_laplacian`, `posterior_weighting`), a set of inductive biases (e.g. `convex_clusters`, `non_convex_capable`, `needs_k`), callables for time and memory complexity in (n, d, k), a hyperparameter space, and whether k is an input. Cards serve three purposes.

- **Theoretical predictions.** Every result row carries three predicted columns (`theoretical_*`: wall time, memory and an ARI upper bound) next to the measured values. Against the earlier timing method, which included scoring (§6), predicted wall time landed within 0.6–1.5× of measured for every algorithm, and the deviations were diagnostic: mean shift's ratio of 0.59 showed that its card's $O(n^2)$ is pessimistic for the bin-seeded implementation. The cards have not yet been recalibrated against the corrected timings.
- **Gap finding.** Given dataset fingerprints (effective dimension, convexity, outlier fraction, density skew and similar), the gap finder scores every card's predicted ARI upper bound and reports regimes where the best card falls below a threshold. Those regimes become new datasets.
- **Mutation.** The mutator applies four operators (`swap_mechanism`, `add_bias`, `crossover`, `stack`) to cards and keeps candidates whose predicted mean ARI beats the best existing card by at least 0.02. The top candidate in a 200-step random search was implemented and benchmarked [8].

The card predictions are an *ordering* heuristic, not a calibrated estimate. The mutator predicted 0.926 ARI for its proposed hybrid, which actually reached 0.747 (§7).

3.7 Validity controls

An earlier version of this paper listed threats to validity without addressing them. Each now has a control in the harness (Table 1), switched on in all configurations used below.

Statistics. With $k \approx 50$ algorithms the Friedman test is over-powered: it says that *some* algorithms differ, not which. The `stats` module adds three things on top of it:

- the Nemenyi critical difference (CD) for average ranks [12];
- Wilcoxon signed-rank tests between pairs of algorithms with Holm step-down correction, run between every pair of the top ten and between the best algorithm and every other;
- percentile bootstrap confidence intervals for each mean, resampling dataset *configurations* so that the five correlated seeds of a configuration move together.

Timing. Only `fit_predict` is timed. Previously the timed window also contained label I/O and metric computation, including the $O(n^2)$ silhouette. An untimed warm-up call per algorithm absorbs one-off costs: the learned selectors train on first use, which had added up to 14 s to the first task. Each run is timed three times and the median is reported (all three are kept). BLAS and OpenMP are pinned to one thread through `threadpoolctl`, and the manifest records CPU model, core counts, memory, package versions, git commit and a hash of the configuration.

Provenance. Every row records `dataset_source` (`real`, `synthetic` or `synthetic_fallback`), and with `strict_provenance` the harness refuses a real dataset whose loader fell back to synthetic data. The UCR loader now downloads the archive's canonical zips, verifies a pinned SHA-256, and concatenates the TRAIN and TEST splits.

Isolation. Large sweeps fail in ways a small grid never shows: a single run can exhaust memory and have the whole sweep killed by the operating system, or block in native code that a Python alarm cannot interrupt. With `resources.isolate` set, each run executes in a child process forked from the warmed-up harness. The child limits its own address space to its baseline plus a budget (3 GB below), and the parent kills it when a per-algorithm time limit expires. Runs that hit a limit are recorded as `Timeout`, `MemoryError` or `Killed` rows with the task key, so a slow or memory-hungry algorithm appears as a did-not-finish result instead of silently disappearing from its mean.

Table 1: Threats to validity and the controls now in the harness.

threat	control
Friedman over-powered; no pairwise evidence	Nemenyi CD, Holm-corrected Wilcoxon, configuration bootstrap CIs
Three seeds per configuration	five seeds; per-configuration seed spread reported
Timed window included scoring and I/O	time `fit_predict` only
One-off training cost in first task	untimed warm-up
Single noisy timing; threads uncontrolled	median of 3; one BLAS/OpenMP thread; hardware in manifest
Real data silently replaced by synthetic	`dataset_source` per row; `strict_provenance`; checksummed UCR zips
Injected outliers scored as a class	`ari_inliers`, `nmi_inliers`
$n \leq 2000$	sweeps to n = 200,000, d = 500, k = 200; logged `max_n` caps
Runaway runs stall or kill the sweep	each run in a forked child with time and memory limits; DNF rows recorded
Selector leakage through sibling seeds	leave-one-dataset-family-out retraining
Selectors and agent trained from git-ignored files	one committed training table; agent rebuilds its traces
Algorithms missing from OOD split	unseen config covers every algorithm
Failed runs silently drop from means	failures root-caused (§6)

Selector leakage. All learned selectors read their training table through one module, which can exclude dataset families. A leave-one-dataset-family-out (LOFO) script retrains every selector without the test family, one family at a time.

3.8 Dashboard and continuous evaluation

A static, dependency-free HTML dashboard served from the repository's `docs/` folder reads the results and trajectory JSON. It shows per-algorithm comparisons, a filterable per-task leaderboard, rank and Friedman tests computed in the browser, and a trajectory viewer. Further panels show the every-dataset run by dataset group, the big-data scaling table, graph clustering at $n = 100{,}000$ and the held-out split. A build script converts any run directory into dashboard JSON, so refreshing the public leaderboard is one command plus a commit.

4 The benchmark suite

Datasets. Generators share one contract, DataSpec $\mapsto (X, y)$, with $y = -1$ for injected outliers and noise. They fall into six groups.

- *Convex and near-convex synthetic:* isotropic blobs, an MDCGen-style mixture [11] with outlier, noise and density knobs, sheared (anisotropic) blobs, imbalanced blobs, hierarchical nested Gaussians, and heavy-tailed Student-t mixtures.
- *Non-convex synthetic:* moons, concentric circles, spirals, Swiss roll, S-curve, rings and mixed shapes.
- *Heterogeneous density and contamination:* varying density and `extreme_outliers`, which uses a tunable outlier extremity.
- *Low rank in high dimension:* an inverse-PCA generator that places cluster means in a low-dimensional latent space and projects them to 50–200 ambient dimensions.
- *Real tabular data:* Iris [29], Wine, Breast Cancer, Digits (projected to 16 dimensions), Olivetti faces, Glass, Vehicle, Segment, Yeast and E. coli [30].
- *Other modalities:* time series (UCR ECG200 and Trace [31], plus a synthetic set) reduced to a 16-dimensional summary of 8 statistics, 4 FFT magnitudes and 4 autocorrelations; graph

nodes (Zachary's karate club [32] and a stochastic block model [33]) reduced to 6 centralities and 10 Laplacian eigenvectors.
- *Adversarial:* `PowerLawStudentT` (anisotropic Student-t components with Zipf-distributed masses, whose first two moments look Gaussian) and `VariableDensityBridges` (dense cores with Cauchy tails joined by thin Poisson bridges, so no single ε works). Both were designed specifically to break the learned selectors.

Algorithms. The registry holds 50 implementations. The classical baselines are k-means with k-means++ seeding [34, 35], mini-batch k-means [7], MapReduce-style parallel k-means [6], BIRCH [36], Ward agglomerative [37], Gaussian mixtures via EM [38], spectral clustering [39], DBSCAN [40], OPTICS [41], mean shift [42], CLARANS [28], Chameleon [43], an unweighted consensus vote, PWCC [5] and S^{5}C [4]. The last four of the thesis's representatives (parallel k-means, mini-batch, PWCC and S^{5}C) cover its four taxonomy categories. The remaining entries are new algorithms described in the companion paper [8]: basis-mixture models, targeted fixes, synthesised pipelines, a graph-native method, ten learned selectors and a behaviour-cloned agent.

Evaluation splits.
- *In-distribution* (`benchmark.paper.demo.yaml`): 28 task variants from 20 dataset families (inverse-PCA restored), seeds $\{1, \ldots, 5\}$, $n \leq 1000$, giving 140 tasks.
- *Leave-one-family-out* (`scripts/router_lofo.py`): the same 140 tasks, with each selector retrained without the family under test.
- *Out-of-distribution* (`benchmark.unseen.yaml`): seven configurations with (n, d, k) tuples, outlier extremities and seeds (100–104) absent from training, e.g. MDCGen at d=25, k=7, moons embedded in d=5, inverse-PCA at d=150, and outlier extremity 10, giving 35 tasks for all 62 algorithms (the 61 above plus Chameleon).
- *Large-n* (`benchmark.scaling.large.yaml`): MDCGen (d=10, k=5) and moons at $n \in \{10^3, 3 \cdot 10^3, 10^4, 3 \cdot 10^4, 5 \cdot 10^4\}$, two seeds, 18 algorithms.
- *Big data* (`benchmark.bigdata.yaml`): MDCGen along three one-factor axes through n=10^4, d=20, k=10, namely $n \in \{2 \cdot 10^3, \ldots, 2 \cdot 10^5\}$, $d \in \{5, \ldots, 500\}$ and $k \in \{3, \ldots, 200\}$, plus a $10^5 \times 100 \times 50$ corner; one seed, 120 s and 3 GB per isolated run, 61 algorithms.
- *Every dataset* (`benchmark.shapes.yaml`): every dataset generator. The twenty shape and mixture generators run at $n = 2000$, both in their native two or three dimensions and padded to $d = 10$ (rings and spirals also at two values of k). Every real dataset runs at close to native size. Three seeds, isolated runs, 63 algorithms, 180 tasks.

5 Results

All numbers below come from the data committed in the repository under `docs/`: the in-distribution, unseen and scaling runs and the LOFO results. The figures are regenerated from those files by the script `make_figures.py` in `paper/scripts`. Every real dataset loaded as real data: 2,135 in-distribution rows are `real` and none is a fallback.

5.1 Overall leaderboard

The in-distribution run covers 61 algorithms on 140 tasks, giving 8,539 result rows. Over the 139 tasks every algorithm completed, the Friedman test gives $\chi^2 = 2700$ ($p < 10^{-300}$). Figure 2 shows average ranks and Figure 3 adds confidence intervals.
- **Nemenyi cannot separate the top of the board.** The CD is 8.7 ranks, and fifteen algorithms are within one CD of the best:
  - seven learned selectors;
  - LMM and spectral clustering;

  - the tri-basis mixtures TBMM and TBMM-FL;
  - the micro-cluster-graph variants `lmm_mcg_dual` and `lmm_mcg`;
  - AURA-v3 and PWCC with diverse bases.
- **Pairwise tests are sharper.** Holm-corrected Wilcoxon tests distinguish 76 of the 96 pairs tested.
  - The best selector beats LMM by 0.051 ARI (Holm $p = 2 \times 10^{-4}$) and the best micro-cluster variant by 0.047 ($p_{\text{Holm}} = 1 \times 10^{-4}$).
  - Selector v3 beats selector v1 ($p_{\text{Holm}} = 0.003$).
  - LMM and spectral clustering are indistinguishable ($p_{\text{Holm}} = 1$).
- **Intervals are wide.** Configuration-level 95% bootstrap intervals are about ± 0.09 ARI. For example, the best selector scores 0.815 [0.727, 0.894] and LMM 0.765 [0.661, 0.857]. Most of this width is *between-configuration* variability, so twenty families are not enough to rank closely matched algorithms with confidence.
- **Seed variability is moderate but uneven.** The mean across-seed standard deviation of ARI within a configuration is 0.045 for LMM, 0.081 for k-means and 0.144 for auto-ε DBSCAN.

5.2 No algorithm dominates

Figure 4 breaks ARI down by dataset family. Among the 51 non-selectors, 35 different algorithms are best on at least one family.

- *Convex-only* methods (k-means family, BIRCH, Ward) score ≈ 0 on circles.
- *Graph-Laplacian* methods (spectral, LMM, AURA) solve circles, but spectral and LMM reach only about 0.63 on moons. The micro-cluster graph methods of the companion paper reach 0.98–0.99 there, close to `louvain_knn` (1.00).
- `louvain_knn` is best on the karate graph (0.73) and Digits (0.55), but weak on imbalanced blobs (0.57) and power-law Student-t mixtures (0.50).
- Gaussian mixtures win Iris (0.90) and Wine (0.58).
- Real time series are hard for everyone. On ECG200 the best method (a mixture on Fourier, Laplace and Hilbert transform features) reaches 0.36 ARI, and on Trace the best reaches 0.66.

5.3 Outlier scoring changes the ranking

Fifteen tasks contain injected outliers (label -1). Plain ARI treats them as an extra true cluster, so an algorithm is rewarded for isolating outliers together and penalised for absorbing them into the nearest cluster. Restricted to inliers, the picture changes:

- LMM rises from 0.733 to 0.990 and spectral from 0.723 to 0.989, while GMM falls from 0.702 to 0.668. The micro-cluster graph methods rise less (`lmm_mcg` from 0.729 to 0.937).
- On these tasks the inlier-only ranking is led by LMM and spectral ($\chi^2 = 380$), whereas the plain-ARI ranking is led by the selectors.

Which convention is right depends on the application. A benchmark should report both, because the choice decides the winner on contaminated data.

5.4 Failure modes are few and recurrent

The algorithm analysis bundled with the repository diagnoses each algorithm along scaling, clean-data quality, outlier robustness and shape adaptability. Six failure modes account for almost every weakness: outlier sensitivity of mean-based updates, non-convex shape failure of Voronoi-cell methods, super-linear scaling, hyperparameter brittleness, local-minimum search, and wrong data assumptions. Because the harness runs originals and variants side by side, it can test whether a diagnosed fix helps. Table 2 reports five one-line fixes from the round in which they were introduced.

Table 2: Diagnosed one-line fixes, mean ARI over the 16-task demonstration sweep (round 2 snapshot, before the validity controls).

original	fixed variant	ARI before	ARI after	Δ
`dbscan`	`dbscan_auto` (k-distance knee)	0.000	0.589	+0.589
`meanshift`	`meanshift_robust` (trimmed bandwidth)	0.470	0.622	+0.151
`clarans`	`clarans_pp` (k-means++ seeding)	0.581	0.656	+0.075
`pwcc`	`pwcc_diverse` (heterogeneous bases)	0.702	0.751	+0.049
`kmeans`	`kmeans_trimmed` (10% trim)	0.701	0.703	+0.003

5.5 Scaling beyond the CI grid

Figure 5 shows the large-n sweep. Above $n = 3{,}000$ on 10-dimensional MDCGen:

- **BIRCH is not linear.** Its log–log slope is 2.1, and it takes 143 s at n=50,000. With the default threshold the CF-tree grows too many leaves in 10 dimensions, so the global step dominates. On 2-d moons it stays linear. Our earlier claim that BIRCH was "the only honest linear scaler" came from the $n \leq 2000$ grid.
- **LMM scales far better than spectral clustering on the same graph Laplacian.** LMM's slope is 1.5 against 2.9 for spectral, giving 6 s against 297 s at n=30,000, with ARI 1.00 for both.
- **Several methods stay near-linear:** mini-batch k-means (slope 0.5), streaming coresets (0.9), FMM (1.0), GMM (1.0) and AURA-v3's Nyström path (1.1).
- **Auto-ε DBSCAN collapses at large** n (ARI 0.01 above $n = 30{,}000$), while `louvain_knn` keeps moons at 1.00 up to $n = 50{,}000$.

Capped runs (Ward and robust mean shift above 10^4, OPTICS above $3 \cdot 10^3$, quadratic methods above $3 \cdot 10^4$) are logged as skips. Across the 5,694 in-distribution runs longer than 10 ms, the median coefficient of variation of the three timing repeats is 5.4% (90th percentile 15.6%). Differences in wall time below about 1.3× should therefore not be read as real.

5.6 Big data: growth in n, d and k

The big-data sweep runs all 61 algorithms with run isolation, 120 s and 3 GB per run. Of 671 runs, 530 finished; 78 hit the time limit, 60 the memory limit and 3 a `max_n` cap. Figure 6 plots wall time along each axis, and the repository's scaling table lists each algorithm's log–log growth exponent in n, d and k.

- **Nineteen algorithms finish** $n = 200{,}000$**.**
  - Fast and exact: mini-batch k-means (3.6 s, ARI 1.00), Trident (3.4 s, 1.00) and GMM (2.8 s, 0.96).
  - The micro-cluster graph method `mcg_cf`: 1.1 s, 1.00.
  - Slower but complete: k-means and FMM (28–37 s).
- **Only 13 finish the** $10^5 \times 100 \times 50$ **corner.** The accurate ones:
  - `mcg_cf`: 1.00 in 1.3 s;
  - Trident: 0.99 in 3.5 s;
  - streaming coresets: 1.00 in 18.5 s;
  - GMM: 0.97 in 64 s;
  - LMM: 1.00 in 102 s.
- **Graph and hierarchical methods are quadratic in** n**.** Ward (exponent 2.36), density peaks (2.29), spectral (2.18) and BIRCH (2.01) never finish beyond $n = 10^4$. The learned selectors inherit their candidates' cost (exponents 1.3–1.8), and none reaches $2 \cdot 10^5$. LMM's exponent is 1.53.
- **Cost in** k **separates the families.** Lloyd-type k-means variants grow with exponents 1.5–1.9

(k-means takes 83 s at $k = 200$). Ward and BIRCH are almost flat in k and reach ARI 1.00 at $k = 200$ in about 4–5 s.

- **Dimension costs memory more than time.** Most exponents in d are below 0.6. The FLH transform-feature mixtures fail at $n \geq 5 \cdot 10^4$ because one feature block allocates an $n \times 300 \times d$ tensor.

The sweep uses one seed per configuration, so it measures cost, not small differences in accuracy.

5.7 Every algorithm on every dataset

The every-dataset run asks a practitioner's question: on data of a given *kind*, which methods work? It runs 63 algorithms on all 36 dataset generators, 11,328 runs in total. Only 12 did not finish, all of them mean shift on the 64-dimensional Digits and 50-dimensional Olivetti faces; they count as ARI 0. Figure 7 summarises it by dataset group.

- **Curved shapes split the field.** On moons, circles, rings, spirals, the S-curve and the Swiss roll:
  - Graph-based methods lead. The four micro-cluster graph methods score 0.80–0.86; spectral clustering and LMM 0.74–0.78.
  - Centroid methods score 0.21–0.26, and the main selectors (v1–v5, v7, v8) 0.31–0.35.
  - Circles are 0.00 for k-means, GMM and selector v3.

  The selectors were trained on families where a centroid method is often right, and they keep choosing one here.
- **Mixtures favour full-graph mixture models.** On mixed shapes, density bridges, nested and heavy-tailed mixtures, TBMM, LMM and spectral clustering lead (0.78–0.79).
- **Convex blobs favour GMM and k-means** (0.79–0.80).
- **Density, imbalance and outliers favour hierarchical summaries.** BIRCH, `mcg_cf` and Ward lead (0.68–0.69).
- **Real data favours the selectors only where they have seen it.** Over all fourteen real datasets the best selector leads (0.49 against 0.40 for GMM, the best single method). Seven of them (Iris, Wine, Breast Cancer, Digits, ECG200, Trace, karate) are in the selectors' training table, at other sizes and dimensions; there the selectors score 0.68 against 0.52. On the seven they have never seen (E. coli, Glass, Yeast, Vehicle, Segment, Olivetti faces, 20 Newsgroups) they score 0.30, level with the best single methods (0.30, streaming coresets). This is the leave-one-family-out result of §5.8 on real data.

The overall order depends on the mix of dataset kinds. Over all 180 tasks the micro-cluster method `lmm_mcg_dual` leads (0.686), and the best selector ranks 16th (0.580). That reverses the in-distribution ranking. A benchmark's headline winner is a statement about its dataset mix, so we report both splits and the per-group view.

5.8 Generalisation of the learned selectors

The selectors are trained on benchmark results. Their in-distribution scores drop only an exact fingerprint match, so sibling seeds of the same configuration stay in the training set. Two stricter evaluations measure the cost of that leakage.

Leave one dataset family out. Retraining each selector without the family under test lowers its mean ARI over the same 140 tasks:

- v1 from 0.788 to 0.728, and v3, v5 and v7 from 0.815 to 0.745;
- v8 from 0.808 to 0.748;
- v4, the most robust, from 0.814 to 0.784.

The losses are concentrated in families whose winner is unusual. When moons is held out, v3 can no longer learn to dispatch to `louvain_knn` and loses 0.70 ARI there; the karate graph costs 0.32. Against LMM and spectral clustering on the same 140 tasks, no LOFO selector differs

significantly (paired Wilcoxon, Holm $p \geq 0.07$). Their mean differences range from -0.037 to $+0.020$.

Unseen configurations. On the out-of-distribution split (Figure 8), the four micro-cluster graph methods lead by mean ARI (0.890–0.925). They are followed by TBMM (0.869), spectral clustering (0.860), LMM (0.858), `louvain_knn` (0.846) and selectors v3, v5 and v7 (0.845). None of the differences among these is significant after Holm correction, and with 35 tasks the CD is 17.6 ranks. These seven configurations are on average easier than the in-distribution families (most algorithms sit above the diagonal in Figure 8), so the split tests transfer of the dispatch rule rather than difficulty.

Summary. In-distribution, the best selector is significantly better than any single algorithm. On families or configurations it has not seen, it is statistically tied with LMM and spectral clustering. Its advantage comes from having seen a family's winner, which is exactly what LOFO removes.

5.9 The self-extending loop

Clustbench's learned selectors read the committed results table when first used, so adding an algorithm and re-running the benchmark retrains them with no code change. In the development history [8], adding the graph-community algorithm `louvain_knn` after the karate graph exposed a gap was enough for every fingerprint selector to dispatch to it there on the next run, raising the best selectors' mean ARI by 0.03–0.04. The LOFO result above is the other side of the same mechanism: a selector can only dispatch to a family's winner if it has seen that family.

6 Corrections to earlier results

The controls changed or invalidated several results reported in an earlier draft of this paper and in the repository's analysis notes.

1. **The UCR time series were synthetic.** Both UCR mirrors in the loader had gone dead (a 404 and a 44-byte page), so ECG200 and Trace had silently been replaced by noisy sinusoids, on which most algorithms scored ARI 1.0. On the real series even the best of the 61 algorithms now benchmarked reaches only 0.36 on ECG200 and 0.66 on Trace. The conclusion that "engineered features make time series easy" does not hold.
2. **BIRCH does not scale linearly** in 10 dimensions beyond $n \approx 3{,}000$ (§5.5).
3. **Selectors do not collapse off-distribution once trained on the broader grid.** The earlier draft reported that the top selectors lost 0.11–0.13 ARI on unseen configurations and that the simplest selector generalised best. Trained on the five-seed grid with inverse-PCA restored, v3 scores 0.845 on the unseen split, level with LMM, and v1 no longer leads. The robust conclusion is the LOFO one: selectors are tied with the best single algorithms on unseen families.
4. **Silent failures.** Missing rows drop out of an algorithm's mean, so we root-caused every logged failure:
   - GMM's covariance went singular on Breast Cancer. This removed GMM, PWCC-diverse and every selector that dispatched to GMM on 4 of 5 seeds. GMM now retries with more regularisation.
   - The unseen configuration wrote `reg_covar: 1e-4`, which YAML 1.1 reads as a *string*, so GMM had never produced an out-of-distribution result.
   - Selectors that dispatched to the consensus ensemble crashed because it had no default base set.
   - Selector v8 trained on a git-ignored file, so in a fresh checkout it trained on nothing.

- The behaviour-cloned agent loaded its checkpoint from a git-ignored directory. This explains its missing round-14 results.
- The results Parquet file failed to write (heterogeneous `extra` column) and the error was swallowed.

5. **Timing included scoring.** Earlier wall times contained label I/O and the $O(n^2)$ silhouette, and the first task of each selector contained its training. Comparisons of those numbers across algorithms with very different runtimes are unreliable.
6. **A metric, not an algorithm, killed the first big-data sweep.** The Dunn index materialised an $m^2 \times d$ distance tensor per cluster pair, about 16 GB at $d = 500$. The operating system's out-of-memory killer then ended the whole benchmark process, and every remaining run of that shard was lost. It now uses pairwise-distance routines (peak about 0.3 GB), and run isolation confines any such failure to one row.

7 Discussion

What the harness made cheap. Re-running every algorithm on every task is a single command, and sharded sweeps merge with one script. The controlled rerun reported here took about two hours on four cores. Most of the corrections above were found by the controls themselves: provenance tags, failure logs that are now read rather than counted, and stricter splits.

Learning beats predicting from biases, within distribution. Card-based predictions order candidate mutations but are not calibrated: the mutator's top proposal was predicted at 0.926 ARI and measured at 0.747. Selectors trained on measured results outperform both bias arithmetic and every single algorithm, but only on families they have seen.

Residual threats.

- *Breadth.* Twenty families and seven unseen configurations give wide intervals. Most conclusions about closely matched algorithms are not significant, and more families, not more seeds, would tighten them.
- *Scale.* The big-data sweep reaches $n = 200{,}000$ on one generator with one seed on a single machine, and distributed algorithms run with one worker. The every-dataset run stays at $n = 2000$.
- *Timing.* Runs share one machine; median-of-3 timing still varies by 5–16%.
- *Hyperparameters.* Algorithms run at fixed, hand-chosen settings; a tuned comparison would change some rankings.
- *Unseen split.* Its configurations are synthetic and easier than the in-distribution families.
- *CI cost.* The public dashboard workflow re-runs the controlled configuration, now 61 algorithms, on every push to the main branch. The big-data and every-dataset results are committed rather than re-run.

8 Conclusion

Clustbench turns "which clustering algorithm should I use?" into a reproducible, re-runnable measurement. It records full resource usage and the step-by-step optimisation trajectory, ships theoretical predictions next to measurements, and feeds its own output to selectors that improve when the registry grows. Adding validity controls changed what we can claim. Learned selection is significantly better than any single algorithm on families it has seen, and statistically tied with the best graph-Laplacian methods on families it has not. The every-dataset run makes the same point at a glance: selectors lead on real data they were trained on, tie on real data they were not, and fail on curved shapes, where graph-based methods are best. Big-data runs need isolation as

much as statistics: one memory-hungry metric was enough to lose a shard. Four earlier results were artefacts of silent data fallbacks, small n, missing rows and an unbounded metric. Future work is more dataset families, tuned-hyperparameter comparisons, larger-n distributed runs, importing external dataset batteries such as Gagolewski's [9], and using the trajectory corpus to train components inside algorithms.

Acknowledgement of AI assistance. The benchmark harness's initial scaffold was written by the author. The algorithm implementations, experiments, analysis scripts and drafts of this paper were produced with an AI coding assistant (Claude Code), directed and reviewed by the author, who takes responsibility for the content.

Availability. Code, configurations, committed results, the live dashboard (https://ashail33.github.io/clustbench/) and the scripts that regenerate every figure in this paper are at https://github.com/Ashail33/clustbench.

References

[1] Jon Kleinberg. An impossibility theorem for clustering. In *Advances in Neural Information Processing Systems*, volume 15, 2002.

[2] Iven Van Mechelen, Anne-Laure Boulesteix, Rainer Dangl, Nema Dean, Isabelle Guyon, Christian Hennig, Friedrich Leisch, and Douglas Steinley. Benchmarking in cluster analysis: A white paper. *arXiv preprint arXiv:1809.10496*, 2018.

[3] Ashail Maharaj. Review of big data clustering methods. MEng (structured) research assignment, Faculty of Engineering, Stellenbosch University, 2024. Supervisor: A.P. Engelbrecht.

[4] Shin Matsushima and Maria Brbić. Selective sampling-based scalable sparse subspace clustering. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

[5] Rasim M. Alguliyev, Ramiz M. Aliguliyev, and Lyudmila V. Sukhostat. Weighted consensus clustering and its application to big data. *Expert Systems with Applications*, 150:113294, 2020.

[6] Weizhong Zhao, Huifang Ma, and Qing He. Parallel k-means clustering based on MapReduce. In *Cloud Computing (CloudCom 2009)*, volume 5931 of *Lecture Notes in Computer Science*, pages 674–679, 2009.

[7] D. Sculley. Web-scale k-means clustering. In *Proceedings of the 19th International Conference on World Wide Web*, pages 1177–1178, 2010.

[8] Ashail Maharaj. Synthesising clustering algorithms from benchmark failure modes: Basis mixtures, region-adaptive pipelines, learned routers and a behaviour-cloned clustering agent, 2026. Companion paper, https://github.com/Ashail33/clustbench/tree/master/paper/algorithms.

[9] Marek Gagolewski. A framework for benchmarking clustering algorithms. *SoftwareX*, 20: 101270, 2022.

[10] Adil Fahad, Najlaa Alshatri, Zahir Tari, Abdullah Alamri, Ibrahim Khalil, Albert Y. Zomaya, Sebti Foufou, and Abdelaziz Bouras. A survey of clustering algorithms for big data: Taxonomy and empirical analysis. *IEEE Transactions on Emerging Topics in Computing*, 2 (3):267–279, 2014.

[11] Félix Iglesias, Tanja Zseby, Daniel Ferreira, and Arthur Zimek. MDCGen: Multidimensional dataset generator for clustering. *Journal of Classification*, 36:599–618, 2019.

[12] Janez Demšar. Statistical comparisons of classifiers over multiple data sets. *Journal of Machine Learning Research*, 7:1–30, 2006.

[13] Milton Friedman. The use of ranks to avoid the assumption of normality implicit in the analysis of variance. *Journal of the American Statistical Association*, 32(200):675–701, 1937.

[14] John R. Rice. The algorithm selection problem. *Advances in Computers*, 15:65–118, 1976.

[15] Pavel Brazdil, Christophe Giraud-Carrier, Carlos Soares, and Ricardo Vilalta. *Metalearning: Applications to Data Mining*. Springer, 2009.

[16] Bernhard Pfahringer, Hilan Bensusan, and Christophe Giraud-Carrier. Meta-learning by landmarking various learning algorithms. In *Proceedings of the 17th International Conference on Machine Learning*, pages 743–750, 2000.

[17] Marcilio C. P. de Souto, Ricardo B. C. Prudêncio, Rodrigo G. F. Soares, Daniel S. A. de Araujo, Ivan G. Costa, Teresa B. Ludermir, and Alexander Schliep. Ranking and selecting clustering algorithms using a meta-learning approach. In *2008 IEEE International Joint Conference on Neural Networks*, pages 3729–3735, 2008.

[18] Daniel G. Ferrari and Leandro Nunes de Castro. Clustering algorithm selection by meta-learning systems: A new distance-based problem characterization and ranking combination methods. *Information Sciences*, 301:181–194, 2015.

[19] Dennis Treder-Tschechlov, Manuel Fritz, Holger Schwarz, and Bernhard Mitschang. ML2DAC: Meta-learning to democratize AutoML for clustering analysis. *Proceedings of the ACM on Management of Data*, 1(2), 2023. SIGMOD 2023, doi:10.1145/3589289.

[20] Prabhant Singh, Pieter Gijsbers, Murat Onur Yildirim, Elif Ceren Gok, and Joaquin Vanschoren. CLAMS: A system for zero-shot model selection for clustering. *arXiv preprint arXiv:2407.11286*, 2024.

[21] Mohammadreza Bakhtyari, Bogdan Mazoure, Renato Cordeiro de Amorim, Guillaume Rabusseau, and Vladimir Makarenkov. ClustRecNet: A novel end-to-end deep learning framework for clustering algorithm recommendation. *arXiv preprint arXiv:2509.25289*, 2025.

[22] Matthias Feurer, Aaron Klein, Katharina Eggensperger, Jost Springenberg, Manuel Blum, and Frank Hutter. Efficient and robust automated machine learning. In *Advances in Neural Information Processing Systems*, volume 28, 2015.

[23] Lawrence Hubert and Phipps Arabie. Comparing partitions. *Journal of Classification*, 2(1): 193–218, 1985.

[24] Alexander Strehl and Joydeep Ghosh. Cluster ensembles — a knowledge reuse framework for combining multiple partitions. *Journal of Machine Learning Research*, 3:583–617, 2002.

[25] Peter J. Rousseeuw. Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, 20:53–65, 1987.

[26] David L. Davies and Donald W. Bouldin. A cluster separation measure. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, PAMI-1(2):224–227, 1979.

[27] J. C. Dunn. A fuzzy relative of the ISODATA process and its use in detecting compact well-separated clusters. *Journal of Cybernetics*, 3(3):32–57, 1973.

[28] Raymond T. Ng and Jiawei Han. CLARANS: A method for clustering objects for spatial data mining. *IEEE Transactions on Knowledge and Data Engineering*, 14(5):1003–1016, 2002.

[29] R. A. Fisher. The use of multiple measurements in taxonomic problems. *Annals of Eugenics*, 7(2):179–188, 1936.

[30] Dheeru Dua and Casey Graff. UCI machine learning repository. University of California, Irvine, School of Information and Computer Sciences, 2019.

[31] Hoang Anh Dau, Anthony Bagnall, Kaveh Kamgar, Chin-Chia Michael Yeh, Yan Zhu, Shaghayegh Gharghabi, Chotirat Ann Ratanamahatana, and Eamonn Keogh. The UCR time series archive. *IEEE/CAA Journal of Automatica Sinica*, 6(6):1293–1305, 2019.

[32] Wayne W. Zachary. An information flow model for conflict and fission in small groups. *Journal of Anthropological Research*, 33(4):452–473, 1977.

[33] Paul W. Holland, Kathryn Blackmond Laskey, and Samuel Leinhardt. Stochastic blockmodels: First steps. *Social Networks*, 5(2):109–137, 1983.

[34] Stuart P. Lloyd. Least squares quantization in PCM. *IEEE Transactions on Information Theory*, 28(2):129–137, 1982.

[35] David Arthur and Sergei Vassilvitskii. k-means++: The advantages of careful seeding. In *Proceedings of the 18th Annual ACM-SIAM Symposium on Discrete Algorithms*, pages 1027–1035, 2007.

[36] Tian Zhang, Raghu Ramakrishnan, and Miron Livny. BIRCH: An efficient data clustering method for very large databases. In *Proceedings of the 1996 ACM SIGMOD International Conference on Management of Data*, pages 103–114, 1996.

[37] Joe H. Ward. Hierarchical grouping to optimize an objective function. *Journal of the American Statistical Association*, 58(301):236–244, 1963.

[38] A. P. Dempster, N. M. Laird, and D. B. Rubin. Maximum likelihood from incomplete data via the EM algorithm. *Journal of the Royal Statistical Society: Series B*, 39(1):1–38, 1977.

[39] Andrew Y. Ng, Michael I. Jordan, and Yair Weiss. On spectral clustering: Analysis and an algorithm. In *Advances in Neural Information Processing Systems*, volume 14, 2002.

[40] Martin Ester, Hans-Peter Kriegel, Jörg Sander, and Xiaowei Xu. A density-based algorithm for discovering clusters in large spatial databases with noise. In *Proceedings of the 2nd International Conference on Knowledge Discovery and Data Mining*, pages 226–231, 1996.

[41] Mihael Ankerst, Markus M. Breunig, Hans-Peter Kriegel, and Jörg Sander. OPTICS: Ordering points to identify the clustering structure. In *Proceedings of the 1999 ACM SIGMOD International Conference on Management of Data*, pages 49–60, 1999.

[42] Dorin Comaniciu and Peter Meer. Mean shift: A robust approach toward feature space analysis. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 24(5):603–619, 2002.

[43] George Karypis, Eui-Hong Han, and Vipin Kumar. Chameleon: Hierarchical clustering using dynamic modeling. *Computer*, 32(8):68–75, 1999.

A Record schema

Trajectory rows add `run_id`, `step_idx`, `cost`, `delta_cost`, `accepted`, `action` and `state`.

field	description
`algo`, `dataset_id`	algorithm and dataset keys
`n_samples`, `n_features`, `k_target`	task shape
`compactness`, `outliers`, `noise`, `density`, `seed`	generator knobs
`wall_time_s`, `rss_delta_mb`	wall-clock time, memory delta
`cpu_user_s`, `cpu_system_s`, `read_bytes`, `write_bytes`	CPU and I/O
`ari`, `nmi`	external validity
`silhouette`, `davies_bouldin`, `dunn`, `compactness`, `separation`	internal validity
`n_clusters_found`, `extra`	output cardinality, algorithm extras
`n_steps`, `trajectory_path`	trajectory length and Parquet path
`theoretical_*`	card-based predictions (wall time, RSS, ARI bound)

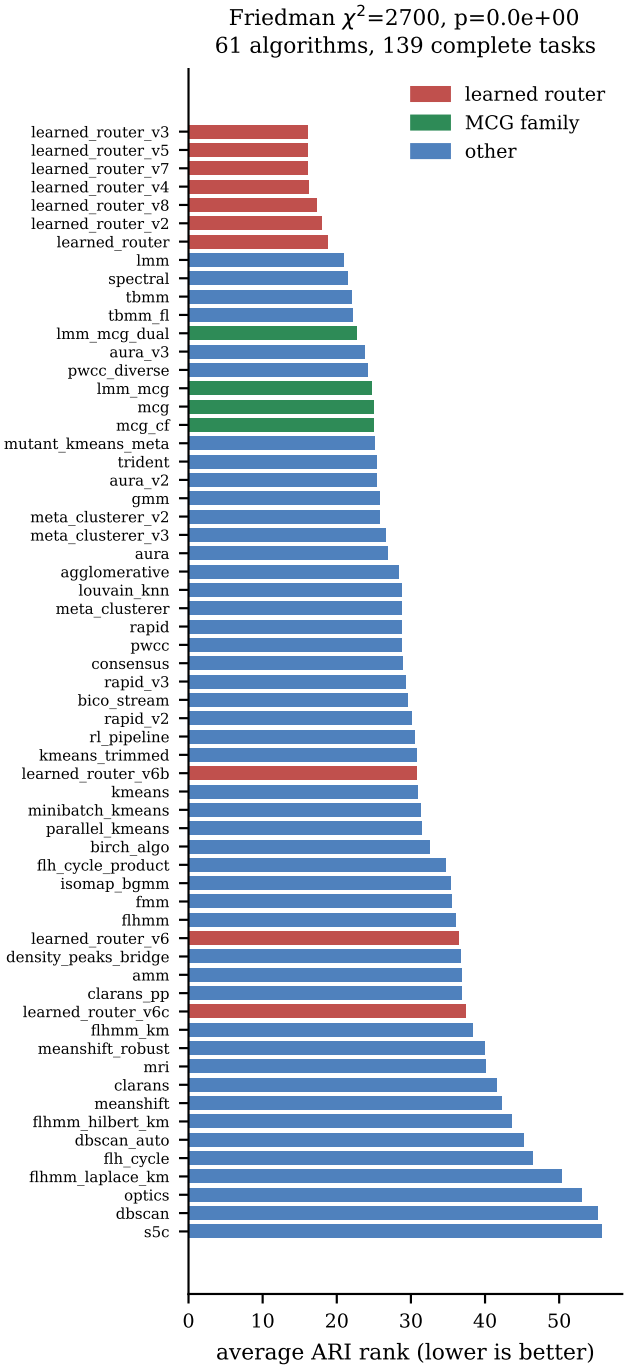


Figure 2: Average ARI rank over the in-distribution split (139 complete tasks). Red bars are learned selectors, which are trained on benchmark results (task-level leave-one-out); green bars are the micro-cluster-graph methods of the companion paper.

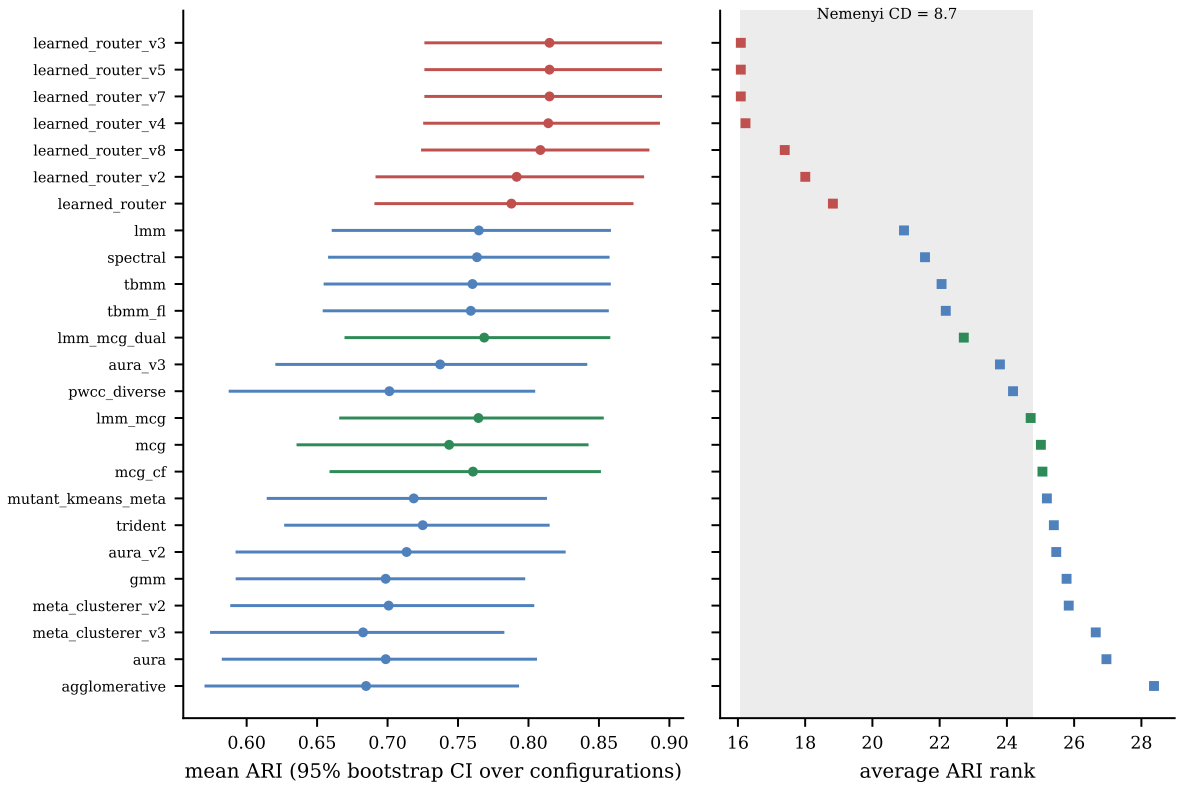


Figure 3: Left: mean ARI with 95% bootstrap intervals that resample dataset configurations. Right: average rank; the grey band is one Nemenyi critical difference from the best.

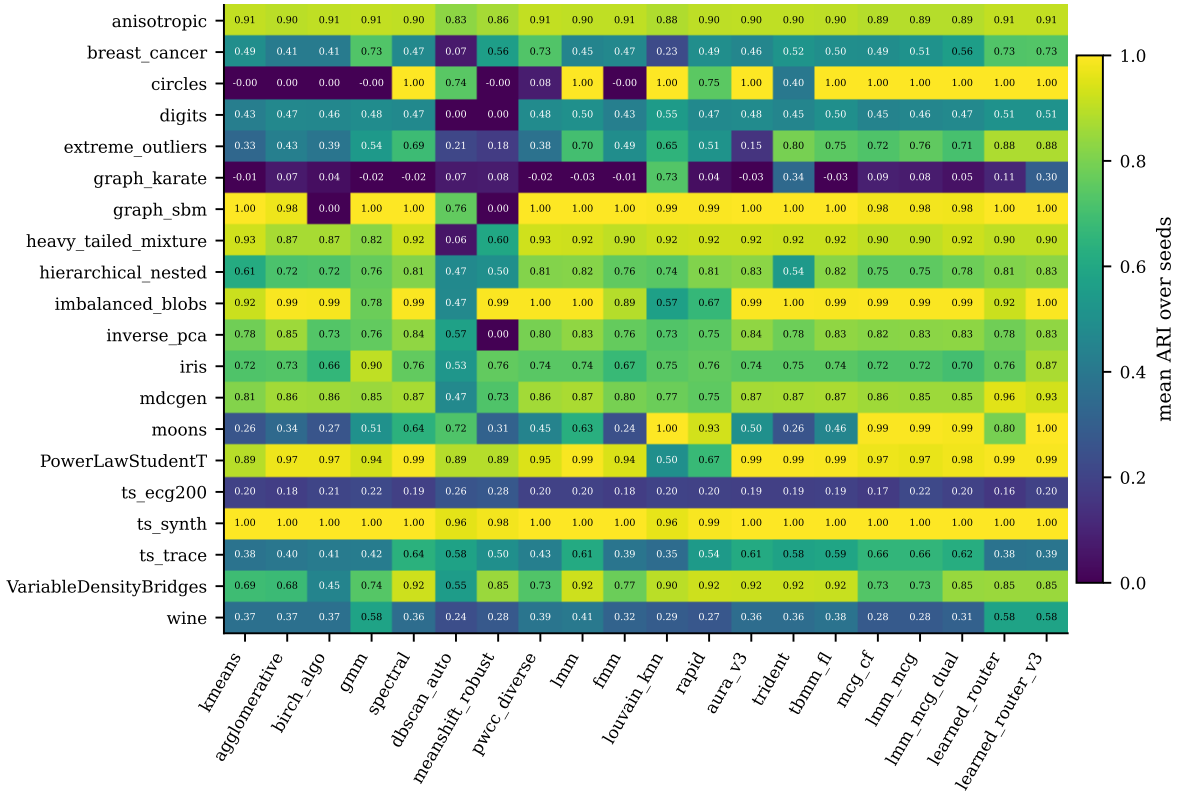


Figure 4: Mean ARI over five seeds for each dataset family (rows) and representative algorithm (columns), in-distribution split.

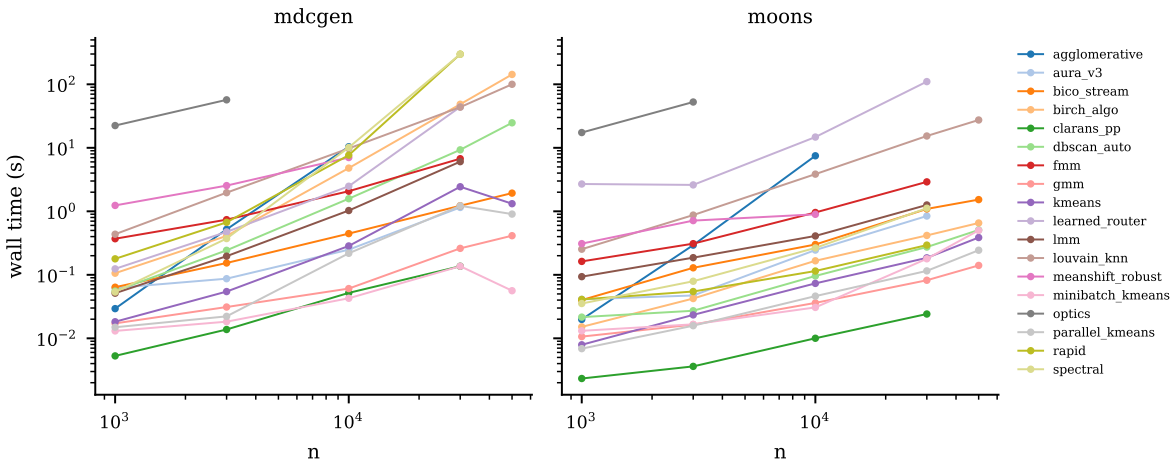


Figure 5: Wall time (fit_predict only, one thread) against n for the large-n sweep, median of two seeds. Lines end where an algorithm's `max_n` cap applies.

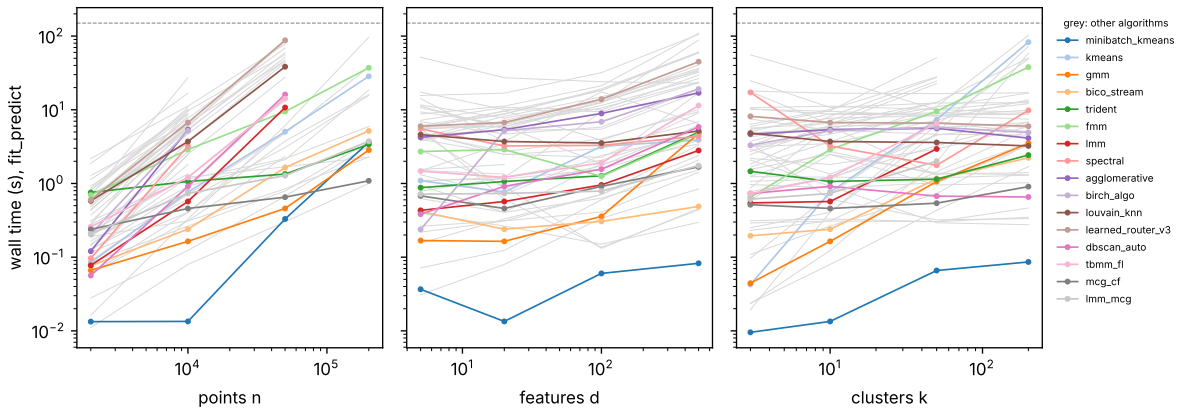


Figure 6: Big-data sweep: wall time (`fit_predict`, one thread) against n, d and k along one-factor axes through n=10^4, d=20, k=10. Coloured lines are representative algorithms; grey lines are the rest. The dashed line is the 120 s limit, and lines stop where runs hit the time or memory limit.

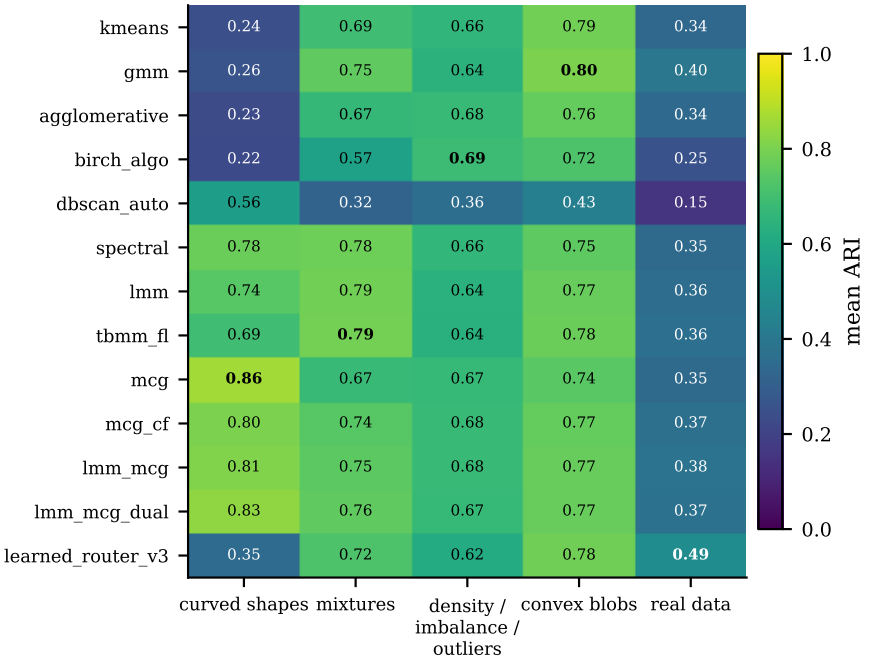


Figure 7: Every-dataset run: mean ARI by dataset group (runs that did not finish count as 0) for representative algorithms. Bold marks the best value per group. Rows are classical baselines, graph and mixture methods, the micro-cluster graph family and the best learned selector.

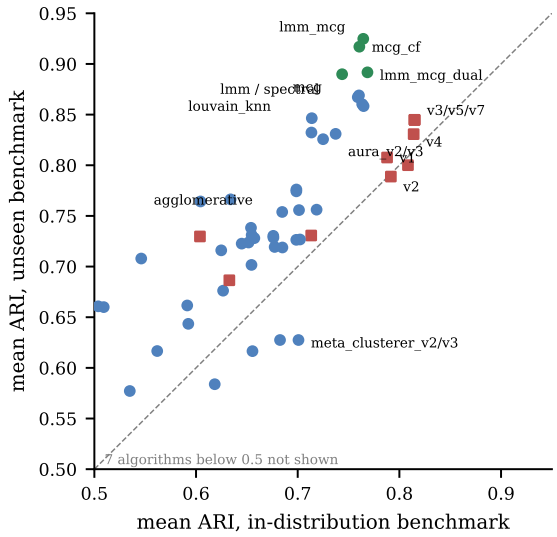


Figure 8: Mean ARI in-distribution vs. on the out-of-distribution split for the 61 algorithms run on both. Red squares are learned selectors, green points the micro-cluster graph methods.